LibraryMS Fullstack

Comprehensive Code Quality & Technical Architectural Review

System Scope: Spring Boot 3.5 & Next.js 16 Fullstack	**Audit Date:** October 2026
Target Artifacts: AuditAspect, DTOs, BookCover, API Client & Services	**Audit Status:** Completed — 14 In-Depth Findings
Severity Overview: 5 High · 6 Medium · 3 Low	**Focus Areas:** AOP Auditing, DTO Validation, UX State, Circulation, N+1

1. Executive Summary

A rigorous, end-to-end review was conducted across the **LibraryMS** fullstack codebase, evaluating the Spring Boot 3.5 backend, REST API layer, JPA repositories, Flyway database schema, and Next.js 16 frontend application. While the system exhibits sound foundational engineering—such as cycle-free DTO projections, BCrypt constraints, and a 14-test regression suite—several **critical omissions, domain logic gaps, and state desynchronization bugs** were identified that impair enterprise production readiness.

Category / Domain	Key Vulnerability / Gap Identified	Severity	Impact Area
Declarative Auditing	Failed operations unlogged; naive entity ID extraction; missing registration/logout audit	HIGH	Compliance & Security
Circulation Lifecycle	No fine settlement mechanism; borrower check ignores open overdue fines	HIGH	Business Rules & Data
Frontend UI State	BookCover loaded/failed state desynchronizes on prop changes; stuck on error	HIGH	Patron Portal UX
Database Performance	N+1 queries in catalog listing (41 queries per page of 20 books)	HIGH	Throughput & Latency
DTO Validation	Unbounded bio strings; DB constraint leakage on genres; author update field clear bug	MEDIUM	API Robustness
Admin Console	AdminUsersPage pagination hardcoded (page=0); no admin role management	MEDIUM	Backoffice Ops
Patron Self-Service	No UI for patrons to edit contact info or self-delete despite API support	LOW	User Experience

2. Declarative Auditing & AOP Flaws (AuditAspect.java)

AuditAspect provides declarative interception via @Around("@annotation(audited)"). However, several crucial flaws undermine its diagnostic and regulatory value:

2.1 Failed Operations Are Completely Unlogged (High Severity):

Inside auditMethod, Object result = pjp.proceed() executes the target service method. If the method throws any exception (such as a ConflictException, BadRequestException, or unhandled runtime failure), the aspect does not catch it. The exception propagates immediately up the stack, **bypassing all audit recording**. In production auditing, attempted illegal actions (e.g. attempting to borrow when suspended, unauthorized record access, deletion conflicts) must be recorded with status FAILURE and the failure reason.

2.2 Fragile Single-Argument Entity ID Extraction (High Severity):

extractEntityId(result, pjp.getArgs()) checks only args[0] when result is void. For methods where the entity ID is in subsequent positions (e.g. linkMember(Long userId, Long memberId), or multi-argument service operations), it either extracts the wrong ID or returns null. Furthermore, reflection invocation of id() and getId() relies on runtime method discovery with swallowed exceptions.

2.3 Omission of Critical Security Actions (Medium Severity):

• AuthService.register(...) creates new user credentials and accounts, yet carries **no audit log**.
• AuthService.logout(...) revokes refresh tokens without logging a logout event.
• Status transitions in BookCopyService.updateCopyStatus(...) only log generic UPDATE_STATUS without tracking old vs new status values.

```
// RECOMMENDED: Resilient Auditing Aspect Pattern
@Around("@annotation(audited)")
public Object auditMethod(ProceedingJoinPoint pjp, Audited audited) throws Throwable {
    Long userId = resolveCurrentUserId();
    String username = resolveCurrentUsername();
    try {
        Object result = pjp.proceed();
        Long entityId = extractEntityId(result, pjp.getArgs());
        auditService.logEvent(userId, username, audited.entityType(), entityId,
                              audited.operation(), "SUCCESS", buildSuccessDetails(audited, result));
        return result;
    } catch (Throwable ex) {
        Long entityId = extractEntityId(null, pjp.getArgs());
        auditService.logAutonomousEvent(userId, username, audited.entityType(), entityId,
                                        audited.operation(), "FAILURE", ex.getMessage());
        throw ex; // Re-throw to maintain error contract
    }
}
```

3. DTO Validation & Domain Invariants (CreateAuthorRequest.java)

3.1 Unbounded Bio String in Author DTOs (Medium Severity):
CreateAuthorRequest validates @NotBlank @Size(max=200) name and @Past birthDate, but String bio has no length restriction. A malicious or malformed client can submit a multi-megabyte string, consuming memory and database buffer cache. Add @Size(max=10000, message="Bio cannot exceed 10,000 characters").

3.2 Inability to Clear Nullable Author Fields (Medium Severity):
In admin/authors/page.tsx, when editing an author and clearing the bio or birth date, the frontend payload sets them to undefined. In JSON serialization, undefined fields are omitted. Because MapStruct's updateEntity ignores omitted values by default, staff cannot clear an existing bio or birth date.

3.3 Database Constraint Leakage on Book Genres (Medium Severity):
CreateBookRequest and UpdateBookRequest accept Set<String> genres without item-level Bean Validation. Passing an

empty string "" or a string exceeding 50 characters bypasses DTO validation and triggers a raw SQL constraint violation (ck_book_genres_not_blank) resulting in an unformatted internal server error.

3.4 Cascading Author Deletion Leaves Books Authorless (High Severity):
In AuthorService.deleteAuthor, deleting an author removes rows from book_authors via database cascade. If an author is the sole contributor to a book, the book is left in the catalog with 0 authors, directly violating the core domain invariant. **Fix:** Verify that no linked books would be left authorless before permitting author deletion.

4. Frontend UI State & API Client Bugs (BookCover.tsx & api.ts)

4.1 BookCover.tsx State Desynchronization (High Severity):

In BookCover.tsx, image failure and load tracking use local component state:
const [failed, setFailed] = useState(false); const [loaded, setLoaded] = useState(false);
When a BookCover component is reused across prop changes (e.g. during catalog pagination, sorting, or modal reuse), **failed and loaded state do not reset**. If the first book image failed, all subsequent books rendered in that slot will remain permanently stuck on failed = true, never attempting to load the new book's cover image.
Fix: Add useEffect(() => { setFailed(false); setLoaded(false); }, [isbn]);.

4.2 Open Library 1x1 Transparent GIF Trap (Medium Severity):

Open Library often serves a 1x1 transparent GIF with HTTP 200 instead of a 404 for missing covers. This triggers onLoad instead of onError, hiding the typeset cloth fallback and leaving an empty box. Check naturalWidth > 1 && naturalHeight > 1 inside onLoad before marking as loaded.

4.3 Missing Fine Settlement API in api.ts (High Severity):

The project documentation claims staff can settle overdue fines in /admin/loans. However, neither frontend/src/lib/api.ts nor LoanController.java exposes any endpoint for settling or paying fines.

4.4 Hardcoded Pagination in AdminUsersPage (Medium Severity):

In frontend/src/app/admin/users/page.tsx, the page index is hardcoded to const page = 0; with no pagination controls. Admins cannot navigate beyond the first 15 user accounts.

5. Circulation Lifecycle & Fine Inconsistencies

5.1 Perpetual Fine Accumulation (High Severity):
When an overdue book is returned in LoanService.returnLoan, loan.setFineAmount(fine) is persisted. However, the database schema contains no fine_settled_at or payment status column. In the patron portal /me, totalFines sums all fines across loan history, causing past, settled fines to display as permanently owed indefinitely.

5.2 Borrow Eligibility Check Ignores Overdue Loans (High Severity):
In LoanService.createLoan, the borrowing guard only verifies:
long openLoans = loanRepository.countByMemberIdAndReturnedAtIsNull(memberId);
If openLoans < maxOpenLoans, the borrow succeeds even if the patron currently holds severely overdue books or thousands in unpaid fines. Standard library management policy requires a hold on checkout if overdue books or unpaid fines exceed a threshold.

6. Performance Inefficiencies & N+1 Queries (BookService.java)

In BookService.listBooks, catalog pagination is implemented as:

```
return bookRepository.findAll(spec, pageable).map(book -> {
    long total = copyRepository.countByBookId(book.getId()); // Query 1
    long available = copyRepository.countByBookIdAndStatus(book.getId(), AVAILABLE); // Query 2
    return bookMapper.toResponseWithCounts(book, total, available);
});
```

Impact: For a standard page size of 20 books, this executes **1 page query + 40 individual count queries = 41 queries** per request. Under concurrent traffic, this causes severe database connection pool exhaustion.
Remediation: Fetch copy counts in a single batch query using GROUP BY copy.book.id or an aggregate projection DTO.

7. Prioritized Remediation Roadmap

Phase	Recommended Action Item	Estimated Effort	Target Files
Phase 1: Critical Fixes	1. Add exception logging to AuditAspect to capture failed operations. 2. Reset failed/loaded states in BookCover.tsx on ISBN change. 3. Add fine_settled_at column and implement POST /loans/{id}/settle-fine.	1 - 2 Days	AuditAspect.java BookCover.tsx LoanService.java
Phase 2: Performance & Validation	1. Eliminate N+1 count queries in BookService.listBooks using batch count query. 2. Guard against author deletion when book would be left authorless. 3. Add @Size validation to bio and validate individual genre strings.	1 Day	BookService.java AuthorService.java CreateBookRequest.java
Phase 3: Backoffice & UX Completion	1. Wire dynamic pagination to AdminUsersPage. 2. Add admin role assignment / promotion endpoint in UserService. 3. Add patron profile update and account self-deletion forms in /me.	1 - 2 Days	AdminUsersPage.tsx UserService.java me/page.tsx